ResolveX Enterprise Multi-Agent Operating System

Comprehensive Architectural Specification, Agent Deep-Dive & Database Engineering Guide

Generated: November 05, 2026 at 12:32 PM | **Version:** 2.0.0 | **Status:** Production Active

Core Framework: LangGraph + FastAPI + React	**Vector Store:** Qdrant Embedded / Remote	**Relational DB:** SQLite / PostgreSQL
Reasoning Model: Qwen-27B (via Groq)	**Governance:** Human-in-the-Loop (HITL)	**Security:** RBAC Matrix + Audit Ledger

1. Agent Hierarchy & Operational Deep-Dive

ResolveX coordinates 5 distinct agent roles configured in a hub-and-spoke multi-agent state graph. Each agent is either a ReAct autonomous reasoning engine equipped with transactional tools and semantic RAG vectors, or a specialized access persona restricted by enterprise security policies.

Agent Name	Role & Archetype	Core Responsibilities	Integrated Tools & Data
Central Supervisor	System Administrator (coordinator.py)	Intent classification, session memory continuity, and global RBAC security gating.	LangGraph StateGraph, structured Pydantic router, fallback heuristics.
HR Agent (HR Sentinel)	People Operations (hr_agent.py)	Employee leave balances, PTO application logic, complete leave history, and PDF audit generation.	check_leave_balance, apply_leave_request, get_leave_history, query_hr_policy (Qdrant), generate_pdf.
Sales Agent (Commercial Copilot)	Revenue & Commercial (sales_agent.py)	Revenue aggregation, order fulfillment, customer transaction logs, pricing playbooks, and financial exports.	get_sales_revenue, get_customer_orders, create_sales_order, query_sales_playbook, generate_excel.
Support Agent (Support Sentinel)	Customer Success & Triage (support_agent.py)	Helpdesk ticket triage, incident tracking, emergency manager escalation, and support SOP retrieval.	get_ticket_status, get_customer_tickets, create_support_ticket, escalate_ticket_to_human, query_support_sop.
Employee Persona (Staff Copilot)	Employee Self-Service (General Staff Portal)	Self-service inquiries: checking own PTO, applying for leave, viewing company policies, and logging tickets.	Restricted scope: Authorized for HR & Support; blocked from Sales commercial data by RBAC gate.

A. Central Supervisor (LangGraph Coordinator)

The Supervisor inspects user intent using structured JSON schema. Crucially, it enforces **Conversational Continuity**: when an agent asks for follow-up details (e.g., number of leave days), the supervisor ensures the user's brief reply ('3 days') routes back to that same agent. Next, it evaluates the **ROLE_PERMISSIONS** matrix. If an unauthorized role attempts to access a restricted domain (e.g. Employee querying Sales revenue), it redirects to the unauthorized_node to block data leakage.

B. HR & People Operations Agent

Manages workforce records and compliance. For leave requests up to 3 days, it auto-deducts the balance and updates records immediately. For extended leaves (> 3 days), it places the request into Pending Approval and inserts an authorization ticket into the Human-in-the-Loop approval queue. It also queries company HR policies via Qdrant semantic search and compiles official downloadable PDF audit reports.

C. Sales & Commercial Copilot

Calculates revenue metrics, queries customer transactions, and processes orders. Any transaction exceeding $1,000 automatically triggers a Sales Director review ticket in the HITL approval system. It also answers discount inquiries using semantic search across sales_playbook.md.

D. Customer Support & Triage Agent

Triages customer issues into Low, Medium, High, or Critical tickets. Sensitive or unresolved incidents are escalated to human operators via escalate_ticket_to_human, creating an urgent manager review item. It consults support_sop.md for SLAs and refund policies.

2. Database Architecture: Where Is The Database & All Files?

ResolveX uses a dual-database architecture: a **Relational Database** (SQLAlchemy ORM with SQLite or PostgreSQL) for structured transactional entities, and a **Vector Database** (Qdrant) for high-dimensional semantic RAG search.

Database Component	Physical Location in Codebase	Engine & Format	Description & Contents
Relational DB (Local)	backend/enterprise.db	SQLite 3 (Automatic Fallback)	Stores users, employees, leave requests, products, orders, support tickets, approvals, and audit logs.
Relational DB (Prod)	Configured in backend/.env: DATABASE_URL=postgresql://...	PostgreSQL 14+ (SQLAlchemy Engine)	Enterprise production database. If unavailable or not configured, automatically falls back to local SQLite.
Vector Store (Qdrant)	backend/qdrant_data/ Collection: enterprise_docs	Qdrant Embedded / Remote Qdrant Server	Stores 384-dimensional dense vector embeddings of chunked markdown and text policies for semantic RAG search.
Source Knowledge Docs	backend/data/docs/	Markdown (.md) & Text (.txt) Files	Raw policy documents: hr_policy.md, sales_playbook.md, support_sop.md.
Generated Reports	backend/reports/	PDF & Excel (.xlsx) Artifacts	Official generated documents downloadable via /api/reports/download/{filename}.

3. How to Add Data to the Database (All Methods)

There are four easy and fully supported ways to add, insert, or ingest data into the ResolveX database ecosystem:

Method 1: Ingesting Documents & Knowledge via the UI Portal (Qdrant + SQLite)

1. Open the web interface at **http://localhost:5173** and sign in as **System Administrator** or any authorized staff member.
2. Navigate to the **Knowledge Documents** module (or click /documents in the sidebar).
3. Click **'New Knowledge Document'** or use one of the pre-built templates (HR, Sales, Support, Compliance).
4. Paste or type your content. When saved, the backend automatically:
   - Saves the source file in backend/data/docs/.
   - Splits the document into semantic markdown chunks.
   - Computes dense 384-dimensional vector embeddings and upserts them into **Qdrant**.
   - Records metadata (title, category, chunk count) in the knowledge_documents relational table.
   - Immediately makes the new knowledge queryable by all AI agents via RAG!

Method 2: Adding Users, Employees & Permissions via Admin Dashboard

1. Sign in as **admin@enterprise.com** and navigate to **Admin & Governance** → **Users & RBAC** (/admin/users).
2. Click **'Add User'** to create an employee or operator account.
3. Assign an enterprise role: admin, hr, sales, support, or employee.
4. Passwords are automatically hashed with secure PBKDF2/bcrypt before committing to the users table.

Method 3: Conversational Tool Ingestion (AI Agent Automated Writes)

You can add operational data directly through conversation with the AI agents in the chat window:

- **Add Leave Request:** Tell the HR Agent: *'Apply for 2 days sick leave for employee 1'* → Writes to leave_requests table.
- **Add Sales Order:** Tell the Sales Agent: *'Place an order for 1x Enterprise AI Suite for customer 1'* → Writes to orders table.
- **Add Support Ticket:** Tell the Support Agent: *'Log a high-priority ticket for customer 2 regarding API latency'* → Writes to support_tickets table.

Method 4: Direct REST API & Python Script Ingestion

Developers can also add data programmatically via the FastAPI endpoints:

• **Add Document via API:** POST http://127.0.0.1:8000/api/documents/create with JSON {"title": "...", "content": "...", "category": "HR"}

• **Direct SQLite Inspection:** Open backend/enterprise.db using any SQLite GUI (e.g. DB Browser for SQLite) or Python: conn = sqlite3.connect('backend/enterprise.db').

• **PostgreSQL Migration:** Set DATABASE_URL=postgresql://user:pass@host:5432/dbname in backend/.env. On startup, SQLAlchemy will automatically run models.Base.metadata.create_all(bind=engine) and provision all tables.

4. Relational Database Schema Reference (models.py)

The following tables are defined in backend/models.py and maintained in enterprise.db:

Table Name	Primary Columns & Data Types	Relationships & Constraints	Business Purpose
users	id (Int, PK), name (Str), email (Str, Unique), hashed_password (Str), role (Str), department (Str), is_active (Bool)	Unique email constraint. Roles: admin, hr, sales, support, employee.	System authentication, RBAC authorization, and user profiles.
employees	id (Int, PK), name (Str), email (Str, Unique), department (Str), leave_balance (Int), is_active (Bool)	1-to-many relationship with leave_requests.	Corporate workforce roster and PTO balance tracking.
leave_requests	id (Int, PK), employee_id (Int, FK), start_date (DateTime), end_date (DateTime), reason (Str), status (Str)	Foreign key → employees.id. Status: Approved, Pending Approval, Rejected.	Ledger of all submitted, approved, and pending employee leaves.
products	id (Int, PK), name (Str), description (Str), price (Float)	1-to-many relationship with orders.	Enterprise product and subscription service catalog.
orders	id (Int, PK), customer_id (Int), product_id (Int, FK), quantity (Int), total_price (Float), status (Str)	Foreign key → products.id. Status: Completed, Cancelled.	Commercial transactional records and revenue tracking.
support_tickets	id (Int, PK), customer_id (Int), subject (Str), description (Str), priority (Str), status (Str)	Priority: Low, Medium, High, Critical. Status: Open, In Progress, Escalated, Closed.	Customer incident triage and SLA resolution queue.
approval_requests	id (Int, PK), title (Str), description (Text), category (Str), requested_by_agent (Str), impact (Str), confidence (Int), status (Str)	Status: Pending, Approved, Rejected. Captures reviewer_name and reviewed_at.	Human-in-the-Loop (HITL) gatekeeper queue for high-impact actions.
knowledge_documents	id (Int, PK), filename (Str), title (Str), category (Str), file_size (Str), chunk_count (Int), uploaded_at (DateTime)	Corresponds to vectorized points in Qdrant collection.	Catalog of enterprise documents ingested into the RAG system.
audit_logs	id (Int, PK), timestamp (DateTime), user_id (Int), user_name (Str), action (Str), resource (Str), details (Text), status (Str)	Append-only tamper-evident audit ledger.	Comprehensive regulatory audit trail of all agent and user actions.

Operational Summary:

- ***Live Application:*** *http://localhost:5173* | ***FastAPI Swagger Docs:*** *http://127.0.0.1:8000/docs*
- ***Default Credentials:*** *All demo accounts (admin, hr, sales, support, employee) share password: password123*
- ***Database File:*** *backend/enterprise.db* | ***Vector Storage:*** *backend/qdrant_data/*
- *This official PDF document is permanently archived in the ResolveX Reports Registry.*